

Handbook of Logic in Computer Science

Volume 3
Semantic Structures

Edited by

S. ABRAMSKY

Professor of Computing Science

DOV M. GABBAY

Professor of Computing Science

and

T. S. E. MAIBAUM

*Professor of Foundations of
Software Engineering*

*Imperial College of Science, Technology and Medicine
London*

Volume Co-ordinator

S. ABRAMSKY

CLARENDON PRESS · OXFORD

1994

Contents

List of contributors

xv

Domain theory

Samson Abramsky and Achim Jung

1	Introduction and Overview	2
1.1	Origins	2
1.2	Our approach	4
1.3	Overview	5
2	Domains individually	6
2.1	Convergence	6
2.2	Approximation	15
2.3	Topology	27
3	Domains collectively	32
3.1	Comparing domains	32
3.2	Finitary constructions	39
3.3	Infinitary constructions	44
4	Cartesian closed categories of domains	52
4.1	Local uniqueness: Lattice-like domains	54
4.2	Finite choice: Compact domains	55
4.3	The hierarchy of categories of domains	62
5	Recursive domain equations	66
5.1	Examples	67
5.2	Construction of solutions	69
5.3	Canonicity	74
5.4	Analysis of solutions	79
6	Equational theories	84
6.1	General techniques	85
6.2	Powderdomains	94
7	Domains and logic	107
7.1	Stone duality	108
7.2	Some equivalences	114
7.3	The logical viewpoint	124
8	Further directions	148
8.1	Further topics in ‘classical domain theory’	148
8.2	Stability and sequentiality	151
8.3	Reformulations of domain theory	152

8.4	Axiomatic domain theory	154
8.5	Synthetic domain theory	155
9	Guide to the literature	156

Denotational semantics 169

R. D. Tennent

1	Introduction	170
1.1	Approaches	171
1.2	An example: binary numerals	172
1.3	Compositionality	175
1.4	Criteria	176
1.5	Overview	178
1.6	Bibliographic notes	179
2	A simple imperative language	180
2.1	Expressions and commands	180
2.2	Assignment commands	185
2.3	Indefinite iterations	187
2.4	Programs	191
2.5	Operational semantics	192
2.6	Programming logic	196
2.7	Non-determinism	203
2.8	Bibliographic notes	205
3	A simple applicative language	205
3.1	Definitions and function applications	206
3.2	Function definitions	212
3.3	Defined notation	214
3.4	Elementary properties	217
3.5	Programming logic	220
3.6	Bibliographic notes	228
4	Recursion	228
4.1	Recursive definitions	228
4.2	Domain-theoretic semantics	231
4.3	Operational semantics	237
4.4	Programming logic	239
4.5	Full abstraction	242
4.6	Untyped procedures	243
4.7	Bibliographic notes	245
5	An Algol-like language I	245
5.1	Syntax	246
5.2	Semantics	250
5.3	Call by value	253
5.4	Programming logic	256

5.5 Bibliographic notes	260
6 An Algol-like language II	260
6.1 Coercions	261
6.2 Local variables	268
6.3 Product types and arrays	270
6.4 Lists	274
6.5 Acceptors	279
6.6 Jumps	282
6.7 Intermediate output	287
6.8 Block expressions	288
6.9 Bibliographic notes	289
7 Possible worlds	290
7.1 Functor–category semantics	291
7.2 Semantic-domain functors	292
7.3 Semantic valuations	295
7.4 Semantics of local variables	298
7.5 Specifications	302
7.6 Non-interference specifications	304
7.7 Semantics of block expressions	308
7.8 Bibliographic notes	311

Algebraic semantics 323

Eric G. Wagner

1 Introduction and motivation	324
1.1 General remarks	324
1.2 Some motivating examples	326
1.3 A notational ‘crisis’	332
2 Algebraic theories, definitions and examples	332
2.1 General remarks	332
2.2 Basic definitions	332
2.3 Algebraic theories as categories	336
2.4 Examples of algebraic theories	337
2.5 Notations and basic identities	341
3 Ordered theories	343
3.1 Definition of ordered and continuous theories	343
3.2 Examples of ordered theories	344
4 Theories with iteration operators	348
4.1 Iteration operators	348
4.2 Iteration, rational, and iterative theories	349
4.3 The ∇ -operator	356
4.4 Iteration operators and flowcharts	356

4.5	Interpretations of flowcharts	364
5	More about iteration operators	366
5.1	Relationships between iteration, rational, and iterative theories	366
5.2	Some identities for iteration operators	372
6	Iteration closure, and normal form theorems	374
7	Free theories and Herbrand interpretations	378
7.1	Free theories	379
7.2	The general case	380
8	Recursive hierarchies	386

The semantics of types in programming languages 395

Carl A. Gunter

1	Introduction	396
2	Types in programming	397
2.1	Higher types	397
2.2	Recursive types	400
2.3	Parametric polymorphism	402
2.4	Subtypes	404
3	Simple types as sets	408
3.1	Types and equations	409
3.2	Sets as a model	412
3.3	Type frames	415
3.4	Completeness for sets	418
4	Simple types as domains	420
4.1	A programming language for computable functions	420
4.2	Operational semantics	422
4.3	Operational equivalence	425
4.4	bc-domains and dI-domains	426
4.5	Full abstraction	427
5	Types as invariants	430
5.1	Run-time safety	430
5.2	Implicit types	434
5.3	Run-time safety for assignments and continuations	441
6	Types as subsets	446
6.1	Untyped λ -calculus	446
6.2	What is a model of the untyped λ -calculus?	448
6.3	What models of the untyped λ -calculus are there?	449
6.4	Inclusive subsets as types	451
6.5	Subtyping as subset inclusion	455
7	Types as partial equivalence relations	458
7.1	Sets as a model of ML_0 types	458

7.2	Another typing system for ML_0	460
7.3	The polymorphic λ -calculus	461
7.4	Sets as a model of polymorphic types?	464
7.5	Simple types as PERs	466
7.6	PERs as a model of polymorphic types	468
8	Conclusion	470
	Index	477